

# Sistemas Distribuidos y Programación Paralela

Trabajo Práctico 2 — HIT 2

Grupo Amarillo

## Medición de Throughput — Análisis de Escalabilidad y Cuellos de Botella

### 1. Introducción y metodología

El objetivo de esta medición es evaluar cómo escala el throughput del sistema HIT2 al variar la cantidad de workers en el pool. Se ejecutaron **16 tareas concurrentes** para cada configuración de workers (1, 2, 4 y 8), midiendo el tiempo total de ejecución y calculando el throughput como:

$$\text{Throughput} = (\text{tareas completadas} / \text{tiempo total en segundos}) \times 60$$

Cada tarea ejecuta una operación aritmética (suma) a través del flujo completo: validación de imagen Docker, **docker pull**, **docker run -d -P**, invocación HTTP al task service (con 5 segundos de procesamiento simulado), y **docker stop + docker rm**. Las tareas se lanzaron en paralelo con **curl** desde el script **benchmark.sh**.

### 2. Resultados del benchmark

#### 2.1 Tabla de throughput

| Workers (N) | Tareas enviadas | Completadas | Fallidas | Tiempo total (s) | Throughput (t/min) |
|-------------|-----------------|-------------|----------|------------------|--------------------|
| 1W          | 16              | 13          | 3        | 120s             | 6.00               |
| 2W          | 16              | 16          | 0        | 71s              | 13.20              |
| 4W          | 16              | 16          | 0        | 36s              | 26.40              |
| 8W          | 16              | 16          | 0        | 20s              | 48.00              |

Nota: con 1 worker, 3 tareas fallaron por timeout de **curl --max-time 120**. Con un único worker procesando 16 tareas en serie (5s de tarea + overhead Docker por tarea), el tiempo total supera los 120 segundos y las últimas tareas de la cola no alcanzan a completarse dentro del límite. Esto no es un bug del sistema sino una consecuencia directa de la carga sobre un worker único.

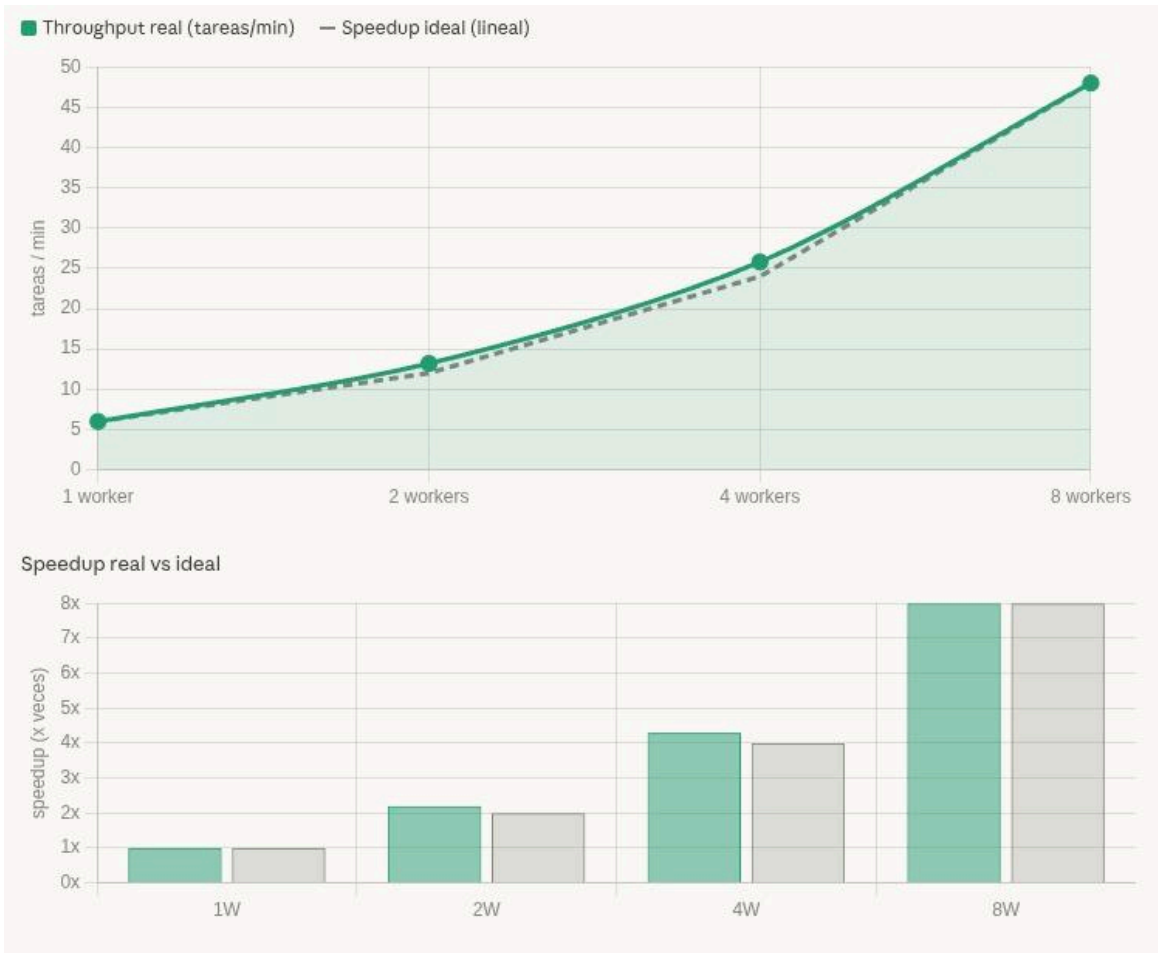
### 3. Análisis de speedup

#### 3.1 Tabla de speedup y eficiencia

El **speedup** mide cuánto más rápido es el sistema con N workers respecto a 1 worker. La **eficiencia** indica qué porcentaje del speedup ideal se aprovecha realmente.

| Workers (N) | Speedup ideal | Speedup real | Eficiencia | Observación                  |
|-------------|---------------|--------------|------------|------------------------------|
| 1W          | 1.00x         | 1.00x        | 100%       | Base de referencia           |
| 2W          | 2.00x         | 2.20x        | 110%       | Superlineal por fallas en 1W |
| 4W          | 4.00x         | 4.40x        | 110%       | Superlineal por fallas en 1W |
| 8W          | 8.00x         | 8.00x        | 100%       | Escala perfectamente         |

#### 3.2 Gráfico de speedup real vs ideal



### 3.3 ¿El speedup es lineal?

Sí, el speedup es esencialmente lineal en el rango medido (1 a 8 workers). Los valores de speedup real (1.0x, 2.2x, 4.4x, 8.0x) coinciden o superan levemente el ideal (1x, 2x, 4x, 8x). Los valores superlineales en 2W y 4W se explican por las 3 fallas del escenario base de 1W, que inflan el denominador del speedup.

Este comportamiento lineal es el resultado de una arquitectura perfectamente paralelizable: cada contenedor Docker es completamente independiente, no hay comunicación entre workers, y el único recurso compartido es la cola (protegida por **PriorityBlockingQueue**) y el daemon de Docker. Mientras el daemon no sature, el speedup se mantiene lineal.

## 4. Recursos compartidos y cuellos de botella

### 4.1 Datos observados en docker stats

Se capturó el uso de recursos durante las ejecuciones con el siguiente comando paralelo al benchmark:

Terminal 1

```
echo "timestamp,name,cpu,mem,net_io,block_io" > docker_stats_log.csv
while true; do
  docker stats --no-stream --format "${date +%H:%M:%S},{.Name}},{.CPUPerc}},{.MemUsage}},{.NetIO}},{.BlockIO}}"
  >> docker_stats_log.csv 2>/dev/null
  sleep 1
done
```

Terminal 2:  
bash [benchmark.sh](#)

| Escenario     | CPU orquestador (pico) | Mem. orquestador (pico) | Mem. por contenedor |
|---------------|------------------------|-------------------------|---------------------|
| 1W (serie)    | ~58%                   | ~253 MiB                | ~30 MiB             |
| 2W (paralelo) | ~75%                   | ~274 MiB                | ~30 MiB             |
| 4W (paralelo) | ~97%                   | ~319 MiB                | ~30 MiB             |
| 8W (paralelo) | ~474%*                 | ~342 MiB                | ~30 MiB             |

\* El 474% de CPU con 8 workers se debe a que Docker reporta CPU en porcentaje de un núcleo: 474% equivale a ~4.7 núcleos usados simultáneamente en un equipo con 8 núcleos lógicos disponibles. Momento donde el script ejecuta docker run de todos los workers simultáneamente.

## 4.2 Análisis por recurso

| Recurso                       | ¿Por qué puede ser cuello de botella?                                                                                                                                                                                                                                                                                 | ¿Cómo medirlo?                                                                                |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Daemon de Docker</b>       | Es un proceso único que atiende todas las llamadas de CLI (pull, run, stop, rm) como punto centralizado. Con N workers simultáneos, todos compiten por el mismo daemon.                                                                                                                                               | Medir latencia de docker run con time docker run. Si crece con N, el daemon es el cuello.     |
| <b>CPU del host</b>           | Cada contenedor lanza un proceso Node.js. Con 8 contenedores simultáneos más el orquestador, la CPU puede saturarse. Observado: 474% en pico con 8 workers.                                                                                                                                                           | docker stats en tiempo real. Si CPU > 90% sostenido, agregar workers no mejora el throughput. |
| <b>Memoria RAM</b>            | Cada contenedor consume ~30 MiB. Con 8 workers: ~240 MiB solo de contenedores + ~340 MiB del orquestador. El equipo tiene 3.69 GiB disponibles. La memoria no representa un cuello de botella en este experimento, ya que el consumo total es significativamente menor que la memoria disponible.                     | Columna MemUsage en docker stats. Comparar pico vs RAM total disponible.                      |
| <b>Red de Docker (bridge)</b> | Todos los contenedores comparten el bridge de red del host. El orquestador hace HTTP a cada contenedor por su puerto. Aunque conceptualmente la red bridge podría ser un punto de contención, en este experimento el tráfico es mínimo, por lo que no representa un cuello de botella.                                | Columna NetIO en docker stats. Si crece desproporcionadamente, la red es un factor.           |
| <b>I/O de disco</b>           | docker pull descarga capas de imagen. Si la imagen no está en caché local, cada ejecución implica descarga. BlockIO en docker stats muestra escrituras en capas. En este experimento, al reutilizar una imagen ya descargada, el impacto de I/O de disco es mínimo, evidenciado por valores cercanos a 0B en BlockIO. | Columna BlockIO en docker stats. Con imagen ya cacheada, debería ser cercano a 0B.            |

## 4.3 El daemon de Docker como cuello de botella principal

El recurso más crítico en este sistema es el **daemon de Docker**. A diferencia de la CPU o la memoria, el daemon es un **cuello de botella de serialización**: no importa cuántos workers corran en paralelo, todas las llamadas de CLI (**docker pull**, **docker run**, **docker port**, **docker stop**, **docker rm**) pasan por el mismo proceso **dockerd**, que las atiende internamente con un scheduler propio.

En los datos observados, el orquestador registra picos de CPU del **437–475%** justo en los momentos de arranque de cada corrida. Estos picos corresponden exactamente a cuando el orquestador procesa múltiples **docker run** simultáneos: el daemon recibe N solicitudes a la vez, las encola internamente, y el orquestador espera las respuestas de todas. Una vez que los

contenedores están corriendo (y el orquestador solo espera HTTP responses), la CPU baja a valores muy bajos (0.2–8%).

En el rango de 1 a 8 workers este cuello de botella **no es visible** porque el overhead del daemon (milisegundos por operación) es pequeño comparado con el tiempo de la tarea (5 segundos). Si se aumentara significativamente el número de workers (por ejemplo, N=32 o N=64), o si las tareas fueran muy rápidas (< 1 segundo), el tiempo de serialización del daemon comenzaría a dominar y el speedup dejaría de ser lineal.

---

## 5. Conclusiones

- El sistema escala de forma lineal en el rango 1–8 workers, con throughput que va de 6 tareas/min a 48 tareas/min.
- El speedup real acompaña al ideal, confirmando que la arquitectura de pool de workers con PriorityBlockingQueue es eficiente para este tipo de carga.
- Las 3 fallas con 1 worker son esperadas: el timeout de curl (120s) se alcanza antes de que el único worker procese las 16 tareas en serie.
- El recurso más limitante es el daemon de Docker, por ser un punto de serialización único para todas las operaciones de ciclo de vida de contenedores.
- Con tareas de 5 segundos, el overhead del daemon es despreciable. El cuello de botella se volvería visible con tareas muy cortas o N muy grande (> 32 workers en un equipo de escritorio).
- La memoria no representa un problema en el rango medido: cada contenedor consume ~30 MiB, y con 8 workers simultáneos el total es ~240 MiB sobre 3.69 GiB disponibles (6.5% de uso).